

```
#!/usr/bin/env python3
```

```
"""
```

Universal JSONL Manifest Compiler (wiki_admin.py)

Crawls repository trees, extracts document headers/first lines, generates SHA-256 hashes, and compiles manifest.jsonl catalogs for fast agent retrieval.

```
"""
```

```
import os
```

```
import json
```

```
import hashlib
```

```
from pathlib import Path
```

```
def compute_sha256(filepath: str) -> str:
```

```
    h = hashlib.sha256()
```

```
    try:
```

```
        with open(filepath, "rb") as f:
```

```
            while chunk := f.read(65536):
```

```
                h.update(chunk)
```

```
    return h.hexdigest()
```

```
    except Exception:
```

```
        return ""
```

```
def build_manifest(doc_dir: str = "./vault", output_path: str = "manifest.jsonl"):
```

```
    records = []
```

```
    base = Path(doc_dir).resolve()
```

```
    for root, _, files in os.walk(base):
```

```
        for file in sorted(files):
```

```
            if file.endswith(("md", ".jsonl", ".py", ".js", ".sh", ".yaml", ".json", ".sql")):
```

```
                full_path = os.path.join(root, file)
```

```
                rel_path = os.path.relpath(full_path, base)
```

```
                summary = "No header found."
```

```
                try:
```

```
                    with open(full_path, "r", encoding="utf-8", errors="ignore") as f:
```

```
                        for line in f:
```

```
                            clean = line.strip()
```

```
                            if clean.startswith("#") or (clean and not clean.startswith("<!--")):
```

```
                                summary = clean.lstrip("#").strip()[:120]
```

```
                                break
```

```
                except Exception:
```

```
                    pass
```

```
                entry = {
```

```
        "file": file,  
        "path": rel_path,  
        "sha256": compute_sha256(full_path),  
        "size_bytes": os.path.getsize(full_path),  
        "summary": summary  
    }  
    records.append(entry)
```

```
with open(output_path, "w", encoding="utf-8") as f:
```

```
    for r in records:
```

```
        f.write(json.dumps(r) + "\n")
```

```
print(f"[✓] Manifest compiled successfully: {len(records)} entries -> {output_path}")
```

```
if __name__ == "__main__":
```

```
    import sys
```

```
    target = sys.argv[1] if len(sys.argv) > 1 else "."
```

```
    out = sys.argv[2] if len(sys.argv) > 2 else "manifest.jsonl"
```

```
    build_manifest(target, out)
```